

M.Sc. in Geodesy and Geoinformatics
with specialization in Hydrography

ULi Motion Compensation

Python Scripts Documentation

Author:

Ahsan Ahammed Ansari

Associated Master's Thesis:

Development and Analysis of Motion Compensation Algorithms
for the Underwater LiDAR ULi

[GitHub Repository Link](#)

Contents

1	Overview	1
1.1	Research Context	1
2	Required Dependencies	2
3	A_json_commander.py	3
3.1	How It Works	3
3.2	Key Function	3
3.3	Usage	3
4	B_trajectory_conversion.py	4
4.1	File Structure	4
4.2	User-Configurable Parameters	4
4.3	Output File	5
4.4	Example of conversion	5
5	C_py_files_gen.py	6
5.1	How It Works	6
5.2	User-Configurable Parameters	6
5.3	Outputs	6
6	D_python_workflow.py	7
6.1	How It Works	7
6.2	User-Configurable Parameters	7
6.3	Rotation Convention	7
7	E_boresight_calibration.py	9
7.1	How It Works	9
7.2	Input Files	9
7.3	GUI Windows	9
7.4	Usage	10
8	F_pulsalyzer_real_time_visualizer.py	11
8.1	How It Works	11
8.2	User-Configurable Parameters	11
8.3	Prerequisites	11
8.4	Usage	11
9	G_pulsalyzer_angles_from_python.py	13
9.1	How It Works	13
9.2	Mathematical Relation	13
9.3	User-Configurable Parameters	13
9.4	Output	13
10	Workflow Guide	15
10.1	Python Motion Compensation Workflow	15
10.2	Pulsalyzer Motion Compensation Workflow	15

1. Overview

This document provides the documentation for Python scripts developed as part of a master's thesis titled "Development and Analysis of Motion Compensation Algorithms for the Underwater LiDAR ULi" at the HafenCity Universität Hamburg. The scripts were developed for the motion compensation and boresight calibration of point clouds acquired with the ULi system, an underwater laser scanning system developed by Fraunhofer IPM.

1.1 Research Context

The ULi system was mounted on a linear motion unit in a recirculating flume at the Bundesanstalt für Wasserbau (BAW), Hamburg, Germany. Data was acquired while the ULi system was moved linearly, which caused motion-induced distortions in the resulting point clouds. To correct these distortions, trajectory data was collected using a Trimble SPS930 total station which actively tracked a prism mounted on the motion unit.

The ULi system captures points in the TAI time standard, and the total station in UTC time standard. A conversion from UTC to TAI is thus required to ensure consistency between the point cloud and trajectory data.

Two independent motion compensation workflows were developed:

1. **Python Workflow** — A fully reproducible workflow developed in Python operating on .las files using rigid body transformations.
2. **Pulsalyzer Workflow** — Using the proprietary *Pulsalyzer* software of the ULi system, adapted to use external trajectory data instead of the onboard INS.

The rigid body transformation applied follows the equation:

$$P_{\text{world}} = R \cdot P_{\text{local}} + T(t)$$

where P_{world} is the point coordinate in the total station (world) frame, P_{local} is the point coordinate in the scanner (local) frame from the .las file, R is a fixed rotation (boresight) matrix defining the relative orientation between the scanner and the total station, and $T(t)$ is the time-interpolated translation vector from the trajectory.

2. Required Dependencies

The scripts were developed using **Python 3.12.10**. The following external Python libraries are also required in addition to the standard library. It is recommended to use a virtual environment to avoid conflicts.

Library	Version	Purpose
laspy	2.6.1	Reading and writing .las point cloud files
numpy	2.3.2	Array operations, linear algebra, numerical computation
open3d	0.19.0	3D point cloud visualization
pandas	2.3.1	Trajectory data manipulation and analysis
scipy	1.16.3	Interpolation, rotation matrix generation
tkinter	-	GUI for boresight calibration tool (Python Standard Library)

Table 2.1: Required Python Libraries

All external libraries can be installed at once using the provided `requirements.txt` file:

```
pip install -r requirements.txt
```

Alternatively, they can be installed individually:

```
pip install laspy==2.6.1 numpy==2.3.2 open3d==0.19.0 pandas==2.3.1 scipy  
==1.16.3
```

Note: tkinter is part of the Python Standard Library and does not require a separate pip installation. However, on Linux it may need to be installed manually (e.g., `sudo apt install python3-tk`). To ensure full compatibility, use **Python 3.12.10**, as the behaviour of standard libraries may vary across Python versions.

3. A_json_commander.py

This is a script provided by Fraunhofer IPM, which can be used to send commands using the JSON interface to programs such as Pulsalyzer. This script thus enables a user to control programs (such as Pulsalyzer) programmatically.

3.1 How It Works

The script connects to a local TCP socket on 127.0.0.1:44444 (JSON_Router) and routes JSON-formatted commands to programs and their endpoints.

For more details regarding the JSON interface, its features and detailed usage, visit the local URL http://localhost:5000/ipm/help/jsoniface_help after launching **JSONServer**, which is provided by Fraunhofer IPM.

3.2 Key Function

```
send_commands(cmds, routing, host="127.0.0.1", port=44444)
```

Parameter	Type	Description
cmds	list	List of command dictionaries (e.g., {"cmd": "reprocess_pointcloud", ...})
routing	list	Routing path to endpoint (see chapter 8 for format)
host	str	IP address of the host (default: "127.0.0.1" (localhost))
port	int	TCP port of the JSON_Router (default: 44444)

3.3 Usage

This script is not run directly. It is to be imported by other scripts, thus controlling programs programmatically. [See [chapter 8](#) for an example]

```
import A_json_commander as json_commander
json_commander.send_commands(cmds, routing)
```

Note: 'A_json_commander.py' must be placed in the same directory as any script that imports it.

4. B_trajectory_conversion.py

This script converts the raw trajectory file obtained from the total station measurements into the format which is compatible by Pulsalyzer software. This converted trajectory is also used as the trajectory input for the Python workflow explained in [chapter 6](#).

4.1 File Structure

The measurements captured by the total station is logged by the Allterra TS Logger software in a .txt file format. Each row in this file contains a time-of-day timestamp (HHMMSS.SSS format in UTC), previously set station coordinates, a horizontal angle (azimuth), a vertical angle (zenith), and a slope distance.

This script reads the raw trajectory file and performs the following operations:

1. Recomputes Cartesian (X, Y, Z) coordinates of tracked points from the polar coordinates (azimuth, zenith, slope distance) with respect to the total station as origin.
2. Converts from left-handed to right-handed coordinate system (negates Y).
3. Applies a lever arm correction along Z to account for the lever arm offset between the tracked prism and the ULi system.
4. Converts time-of-day timestamps to seconds since UNIX epoch (1 January 1970 00:00:00 UTC) and adjusts for the UTC-to-TAI time standard difference.
5. Outputs the trajectory data in tab-delimited Pulsalyzer format.

4.2 User-Configurable Parameters

Variable	Default	Description
Year, Month, Day	'2025', '08', '06'	Date of data collection
x_rot, y_rot, z_rot	0, 0, 0	Scanner orientation (in degrees)
utc2tai_offset	37	UTC-to-TAI offset in seconds
prism_to_scanner_lever_arm	1.65	Vertical distance (m) from prism to ULi lens
input_trajectory_file	R"..."	Full path to raw .txt trajectory file (e.g.: R"C:\Folder \file .txt")

4.3 Output File

The output is a tab-delimited .txt file containing the processed trajectory with the following columns and a corresponding header:

Column	Unit	Description
time	seconds (s)	Timestamps since UNIX epoch (including UTC-to-TAI offset)
x	metres (m)	Cartesian X coordinate in the right-handed total station frame
y	metres (m)	Cartesian Y coordinate in the right-handed total station frame
z	metres (m)	Cartesian Z coordinate in the right-handed total station frame with lever arm correction
roll	degrees (°)	Scanner roll orientation (Default 0.00°)
pitch	degrees (°)	Scanner pitch orientation (Default 0.00°)
yaw	degrees (°)	Scanner yaw orientation (Default 0.00°)

The output file is saved in the same directory as the input trajectory file with the following naming convention:

converted_{Year}_{Month}_{Day}_offset_{utc2tai_offset}_traj_{x_rot}_{y_rot}_{z_rot}.txt

where,

- Year, Month, Day are the date of data collection
- utc2tai_offset is the UTC-to-TAI offset in seconds
- x_rot, y_rot, z_rot are the scanner orientation offsets

4.4 Example of conversion

Raw trajectory

```
101533.231 548775.866 5937527.313 10.950 208.58550 99.53554 35.764
101533.281 548775.866 5937527.312 10.950 208.58530 99.53544 35.763
101533.331 548775.866 5937527.313 10.950 208.58550 99.53534 35.764
101533.382 548775.867 5937527.313 10.950 208.58570 99.53534 35.764
101533.433 548775.866 5937527.313 10.950 208.58560 99.53534 35.764
```

Converted trajectory

```
time x y z roll pitch yaw
1754475370.231000 -35.438322 4.808424 -1.389078 0.000 0.000 0.000
1754475370.281000 -35.437346 4.808178 -1.389029 0.000 0.000 0.000
1754475370.331000 -35.438321 4.808424 -1.388966 0.000 0.000 0.000
1754475370.382000 -35.438306 4.808535 -1.388966 0.000 0.000 0.000
1754475370.433000 -35.438314 4.808479 -1.388966 0.000 0.000 0.000
```

5. C_py_files_gen.py

This script downsamples a .las point cloud to 10% and 1% of the original dataset. The downsampled point clouds are saved as both .las and .csv files. The .csv files are used as input for the boresight calibration GUI ([chapter 7](#))

5.1 How It Works

The .las file is read using the laspy Python library. Downsampling is done by retaining every n^{th} point from the input file - every 10th point for 10% downsampling, and every 100th point for 1% downsampling. The retained points along with all their associated attributes are written to a new .las file using laspy.

A .csv file is then generated from each downsampled .las file using numpy. The .csv files contain only the fields gps_time, x, y, z.

5.2 User-Configurable Parameters

Variable	Default	Description
las_file	R"..."	Full path to the input .las file (e.g.: R"C:\Folder \file .las")

5.3 Outputs

All outputs are saved to the same directory as the input .las file.

- 10%_<filename>.las: 10% downsampled point cloud
- 1%_<filename>.las: 1% downsampled point cloud
- 10%_<filename>.csv: CSV with columns: gps_time, x, y, z
- 1%_<filename>.csv: CSV with columns: gps_time, x, y, z

6. D_python_workflow.py

This is the script for the Python motion compensation workflow. The motion compensation is done by applying a rigid body transformation (rotation + time-interpolated translation) to all points in an input .las file.

6.1 How It Works

This script corrects the motion-induced distortions by:

1. Loading the distorted .las file.
2. Loading the converted Pulsalyzer format trajectory file and removing duplicate timestamps for linear interpolation.
3. Verifying that the .las file timestamps are within the input trajectory time range, if not, stop execution.
4. The trajectory is filtered to the corresponding time window of the input .las file.
5. Checking trajectory linearity with the condition $R^2 \geq 0.999$. If condition fails, stop execution.
6. Linear interpolation functions are then built for x , y , and z as a function of time.
7. Building a rotation matrix from user-defined boresight Euler angles (intrinsic XYZ).
8. Processing the .las file in chunks to manage memory.
9. Rotating each point in a chunk using the rotation matrix and adding the linearly interpolated position from the linear interpolation functions.
10. Each chunk is written incrementally to an output .las file.

6.2 User-Configurable Parameters

Variable	Default	Description
input_las_file	R"..."	Path to distorted .las file (e.g.: R"C:\Folder \file .las")
input_pulsalyzer_trajectory_file	R"..."	Path to converted trajectory .txt file (e.g.: R"C:\Folder \file .txt")
x_rot, y_rot, z_rot	0	Boresight rotation angles in degrees
chunk_size	1000000	Points processed per iteration

6.3 Rotation Convention

The rotation matrix is built using the **Intrinsic XYZ** rotation convention.

The module `scipy.spatial.transform` of the Python library `scipy` is used for building the rotation matrix. The method used for building the rotation matrix from input angles in degrees following the intrinsic XYZ rotation convention is:

```
Rotation.from_euler('XYZ', [x_rot, y_rot, z_rot], degrees=True).as_matrix()
```

For more information on the usage of the module and rotation convention notations, refer to the documentation available at:

<https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.spatial.transform.Rotation.html>

The boresight angles (obtained from the Boresight Calibration ([chapter 7](#))) used in the thesis for the Python workflow were:

$$x_{rot} = 2.5^\circ, \quad y_{rot} = 184^\circ, \quad z_{rot} = 48^\circ$$

7. E_boresight_calibration.py

This script is an interactive GUI application for visually determining the boresight rotation angles. It uses Open3D for 3D visualization and Tkinter for the control panel. A binary (.exe) file of this script is available under releases in the GitHub repository.

7.1 How It Works

The boresight calibration process involves adjusting three rotation angles (Roll/X, Pitch/Y, Yaw/Z) until the point cloud appears geometrically correct. The tool applies the rigid body transformation in real time as sliders are moved. The angles can also be adjusted as text inputs near the sliders in the control panel.

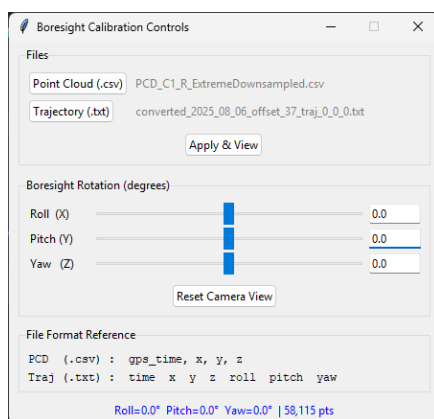
7.2 Input Files

File	Format	Columns Required
Point Cloud	.csv	gps_time, x, y, z
Trajectory	.txt	time, x, y, z, roll, pitch, yaw

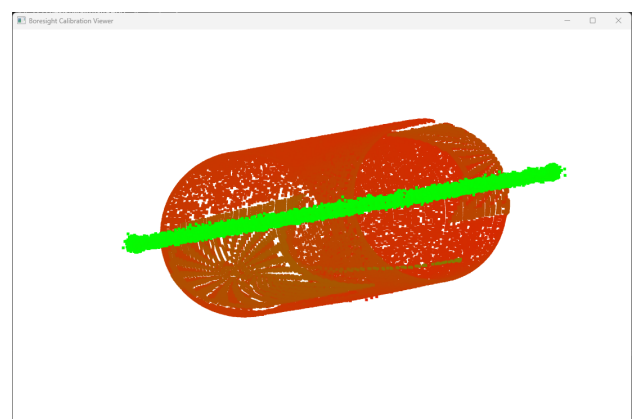
7.3 GUI Windows

This application opens two windows:

1. **Boresight Calibration Controls** (Tkinter) - Main control panel.
2. **Boresight Calibration Viewer** (Open3D) - 3D visualization window.



(a) Boresight Calibration Controls



(b) Boresight Calibration Viewer

Figure 7.1: Boresight Calibration Windows

7.4 Usage

1. Click "Point Cloud (.csv)" → select the downsampled file obtained from Script C ([chapter 5](#)).
2. Click "Trajectory (.txt)" → select the Pulsalyzer format trajectory file obtained from Script B ([chapter 4](#)).
3. Click "Apply & View" → loads files and shows initial point cloud.
4. Use the following mouse controls to navigate the 3D viewer:
 - **Left mouse button + drag** - rotate the view
 - **Middle mouse button + drag (or Ctrl + left click and drag)** - pan the view
 - **Scroll wheel** - zoom in/out
5. "Reset Camera View" button can be used to reset the camera view and centre the point cloud.
6. Adjust Roll (X), Pitch (Y), and Yaw (Z) sliders (Text inputs can also be used to set the values).
7. Note final values for use in the Python workflow ([chapter 6](#)).

The boresight angles obtained for the thesis were:

$[2.5^{\circ}, 184^{\circ}, 48^{\circ}]$

8. F_pulsalyzer_real_time_visualizer.py

This script sends rotation commands to Pulsalyzer using the `A_json_commander.py` script, thus it can be used to visualize the effect of rotation on the Pulsalyzer rendered point cloud in real-time.

8.1 How It Works

Pulsalyzer accepts a scanner transformation command (`set_scanner_transformation_deg`) via its JSON WebServer. This script sends the transformation command along with `reprocess_pointcloud` and `redraw_pointcloud` commands through `A_json_commander.py`, incrementally updating rotation angle along one axis and triggering a reprocess and redraw of the point cloud in Pulsalyzer.

The commands are sent through a `send_silent()` function which suppresses terminal output from `A_json_commander.py`, keeping the terminal clean during the loop.

8.2 User-Configurable Parameters

Variable	Default	Description
<code>routing1</code>	<code>[...]</code>	Routing path to the Pulsalyzer endpoint; must include the correct hostname
<code>x_rot, y_rot, z_rot</code>	<code>0, 0, 0</code>	Initial rotation angles (degrees)
<code>start, stop</code>	<code>0, 10</code>	Start and end values of the angle increment range

8.3 Prerequisites

- JSONServer and Pulsalyzer must be running, with Pulsalyzer having the required `.lidar` file and its corresponding trajectory loaded and processed.
- '`A_json_commander.py`' script must be in the same directory as this script.
- The hostname must match the device name visible in the Pulsalyzer WebServer interface (or `hostname` command in command prompt).

8.4 Usage

The script applies the initial rotation angles, pauses for a keypress, then enters an infinite loop iterating through the angle increment range. The loop restarts when the range is done. Press `Ctrl+C` to stop the loop.

By default, the increment is applied to the Z-axis rotation, modify as needed for other axes. The update rate of point cloud in the 3D viewer of Pulsalyzer depends on the `point_step` parameter of commands `reprocess_pointcloud` and `redraw_pointcloud`.

This script was used in the thesis to identify the hidden rotation angle applied internally by Pulsalyzer. By iterating through a range of rotation angles (-180° to 180°) about the X and Z axes and observing

the rendered point cloud in real-time, the angles at which the Pulsalyzer output visually matched the Python translation-only reference output was identified as 180° , 0° , 90° for X, Y, and Z axes respectively. This hidden rotation matrix was then used to derive the mathematical relationship between the Python and Pulsalyzer boresight angles.

9. G_pulsalyzer_angles_from_python.py

This script calculates the equivalent rotation angles for Pulsalyzer that produce the same point cloud output as the Python workflow, given the boresight calibration angles obtained from Script E ([chapter 7](#)). This calculation is necessary because:

1. Pulsalyzer has a hidden internal rotation
2. Python uses the **Intrinsic XYZ** rotation convention while Pulsalyzer uses **Extrinsic XYZ**. The same set of angles produces a different rotation matrix under each convention.

9.1 How It Works

The hidden internal rotation matrix of Pulsalyzer is first constructed from the identified rotation angles $[180^\circ, 0^\circ, 90^\circ]$ using Extrinsic XYZ convention via `scipy`. The rotation matrix corresponding to the Python boresight angles is then constructed using Intrinsic XYZ convention. The equivalent rotation matrix for Pulsalyzer is computed, and the corresponding Extrinsic XYZ Euler angles are extracted from it.

9.2 Mathematical Relation

Let:

- R_{Python} = rotation matrix from Python boresight angles (Intrinsic XYZ)
- R_{internal} = Pulsalyzer hidden rotation matrix (Extrinsic XYZ: $180^\circ, 0^\circ, 90^\circ$)
- R_{user} = equivalent rotation matrix to be passed to Pulsalyzer (Extrinsic XYZ)

For both workflows to produce the same output:

$$R_{\text{user}} = R_{\text{internal}}^T \cdot R_{\text{Python}}$$

The equivalent Pulsalyzer angles are then extracted from R_{user} using Extrinsic XYZ decomposition.

9.3 User-Configurable Parameters

Variable	Default	Description
X, Y, Z	2.5, 184, 48	Boresight angles (degrees) from Python workflow

9.4 Output

The script prints the equivalent Pulsalyzer angles to the terminal. For the Python boresight angles of $2.5^\circ, 184^\circ, 48^\circ$, the output is:

```
For rotation angles of 2.5, 184, 48 in Python,  
Pulsalyzer angles: rx=1.3, ry=4.53, rz=-42.04
```

These angles are to be passed to Pulsalyzer using the `set_scanner_transformation_deg` command via the WebServer or using `A_json_commander.py` to obtain the same point cloud transformation as the Python workflow.

10. Workflow Guide

10.1 Python Motion Compensation Workflow

1. **Export uncorrected .las from Pulsalyzer:** Load the .lidar file in Pulsalyzer without providing a trajectory file and export it as .las using the Export LAS option in the Point Cloud window.
2. **Run Script B** to convert the raw trajectory to Pulsalyzer format.
 - Set Year, Month, Day to match the scan date.
 - Set prism_to_scanner_lever_arm to the actual measured distance.
 - Run and verify output .txt file is saved to the same folder as the input trajectory file.
3. **Run Script C** to generate a 1% downsampled CSV of the point cloud for use in the boresight calibration tool.
4. **Run Script E** to determine boresight angles:
 - Load the downsampled .csv file from step 3 and the converted trajectory file .txt from step 2.
 - Adjust sliders until the point cloud is geometrically correct.
 - Note the Roll, Pitch, Yaw values.
5. **Run Script D** to apply final motion compensation:
 - Set angles to the values from Step 4.
 - Set the input .las file path and the converted trajectory .txt path.
 - Run and verify the output python_transformed_*.las file.
6. **Open in CloudCompare** to inspect the result.

10.2 Pulsalyzer Motion Compensation Workflow

1. **Run Script B** (same as Python Workflow Step 2) to convert the raw total station trajectory to Pulsalyzer format.
2. **Run Script G** to convert Python boresight angles to Pulsalyzer equivalent angles.
3. **Load .lidar file in Pulsalyzer**
4. **Load the Pulsalyzer format trajectory file**
5. **Apply the Pulsalyzer angles** from Step 2 using the set_scanner_transformation_deg command via the JSON WebServer or Script A.
6. **Process and export the result** as .las file.
7. **Open in CloudCompare** to inspect and compare with the Python workflow output